Lab 4 · Guía del estudiante · Seguridad por diseño

Nov 12, 2026 · @jamil

Práctica opcional en casa, sin nota, sobre el repositorio `mercaplus-legacy`. Respondes al informe de un auditor con un modelo de amenazas, la corrección probada de una inyección SQL y controles de mínimo privilegio. Tiempo estimado: 3 a 4 horas.

El encargo

Una auditoría externa le envió a MercaPlus este informe:

> "El buscador de productos acepta `' OR '1'='1` y devuelve todo el catálogo. La aplicación se conecta a la base con el usuario administrador, la llave de la pasarela está en el código y cualquiera puede cambiar precios por la API."

El CTO te pide responder con tres cosas: un modelo de amenazas que muestre que entiendes el riesgo completo (no solo lo que vio el auditor), la corrección demostrada de la inyección SQL y los controles de mínimo privilegio para cuando la aplicación llegue a AWS.

El laboratorio tiene dos partes y conviene hacerlas en este orden: **primero el modelo de amenazas** (Parte 1), porque es el que dice qué corregir, y **después el código** (Parte 2).

Conceptos de la clase 4 que vas a usar: Security by Design, threat modeling, STRIDE, validación en capas y mínimo privilegio.

Antes de empezar

- [ ] JDK 21, Maven 3.9 o superior, curl y un IDE (IntelliJ IDEA o VS Code).
- [ ] OWASP Threat Dragon, en su versión web o de escritorio (gratuita).
- [ ] Tu copia de `mercaplus-legacy` en un repositorio privado de GitHub, en una rama nueva: `git checkout -b lab4`.
- [ ] La aplicación arranca: `mvn spring-boot:run` y `localhost:8080/api/productos` devuelve 8 productos.
- [ ] Una carpeta para los entregables de consultoría: `consultoria/lab4/`.

Archivos del legado que vas a revisar: `controller/ProductoController.java`, `util/PasarelaPagosClient.java`, `model/Pedido.java` y `src/main/resources/application.properties`.

Si usas un asistente de IA, decláralo en el README: qué herramienta y para qué.

Parte 1 · Modelo de amenazas

El modelo se hace antes de tocar el código: sirve para decidir qué corregir primero. Usa como flujo la compra de un cliente: busca un producto, crea el pedido y paga.

Paso 1 · Diagrama de flujo de datos (DFD)

1. En Threat Dragon crea un modelo nuevo llamado "MercaPlus · compra" y un diagrama STRIDE.
2. Dibuja los **actores externos**: el cliente web y el equipo de precios.
3. Dibuja los **procesos y almacenes**: la API de MercaPlus y la base de datos.
4. Dibuja el **sistema externo**: la pasarela de pagos PagaYa.
5. Une todo con **flujos de datos** y ponle nombre a cada uno: búsqueda, pedido, cobro, respuesta del pago, consultas a la base.
6. Marca las **fronteras de confianza**: los lugares donde cambia el nivel de confianza. Pregúntate por dónde entra lo que no controlas.
7. Guarda el modelo como `consultoria/lab4/threat-model.json` (Threat Dragon lo exporta en JSON).

Pista: toda flecha que cruza una frontera de confianza es un lugar donde buscar amenazas.

Paso 2 · Tabla STRIDE

1. Para cada flujo que cruza una frontera, recorre las 6 letras: **S (suplantación), T (manipulación), R (repudio), I (divulgación de información), D (denegación de servicio) y E (elevación de privilegios)**. Pregunta para cada una: ¿puede pasar aquí?
2. Encuentra en el código cada hallazgo del auditor y anótalo con la clase o el archivo donde está.
3. Agrega **amenazas que el auditor no mencionó**. Un buen modelo va más allá del informe.

4. Califica cada amenaza: probabilidad de 1 a 3, impacto de 1 a 3 y riesgo = probabilidad × impacto.
5. Ordena la tabla por riesgo, de mayor a menor.
6. Escribe una mitigación concreta y un dueño (desarrollo, arquitectura o DevSecOps) para cada una.

Guárdala en `consultoria/lab4/stride.md` con estas columnas, y **mínimo 8 amenazas**:

#	Amenaza	Dónde (clase o archivo)	STRIDE	Probabilidad (1–3)	Impacto (1–3)	Riesgo	Mitigación	Dueño
1								

Paso 3 · Política IAM de mínimo privilegio

El servicio de pedidos va a correr en AWS como una tarea de ECS Fargate. Escribe en `consultoria/lab4/iam-policy.json` la política que le permite **solo** dos cosas:

1. Leer un único secreto de AWS Secrets Manager: `mercaplus/pagaya-api-key`.
2. Escribir registros únicamente en su grupo de logs de CloudWatch: `/ecs/pedidos`.

Reglas: ningún `*` en las acciones y ningún `"Resource": "*"`. Usa una cuenta y una región de ejemplo en los ARN.

Para pensar: el ARN de un secreto en Secrets Manager termina con caracteres aleatorios que no conoces de antemano; ¿cómo lo escribes? Y la política, ¿va en el rol de tarea o en el rol de ejecución de ECS? Explica tu respuesta en un comentario del entregable.

Paso 4 · Usuario de base de datos

En `consultoria/lab4/db-usuario.sql` define, para PostgreSQL, el usuario `mercaplus_app` con el que se conectará la aplicación:

- Solo `SELECT`, `INSERT` y `UPDATE` sobre las tablas que usa la aplicación.
- Sin `DELETE`, `DROP`, `ALTER` ni `TRUNCATE`, y sin acceso a otros esquemas.
- La contraseña no se escribe en el script: indica de dónde vendría.

Para saber qué tablas usa la aplicación, revisa las entidades de `model/` o la consola de la base.

Parte 2 · Código

Trabaja en tu rama `lab4` y haz un commit por paso, para que se vea el orden: ataque, prueba en rojo, corrección en verde.

Paso 5 · Reproducir el ataque

1. Levanta el legado con `mvn spring-boot:run`.
2. Una búsqueda normal: `curl -s -G localhost:8080/api/productos/buscar --data-urlencode "nombre=cafe"`. Debe devolver un producto.
3. El ataque: `curl -s -G localhost:8080/api/productos/buscar --data-urlencode "nombre=' OR '1'='1"`. Anota cuántos productos devuelve.
4. Mira la consola de la aplicación: el legado imprime el SQL que ejecuta. Copia la consulta que se armó y explica en una frase por qué devuelve todo.
5. Revisa los otros hallazgos del auditor y déjalos documentados en tu tabla STRIDE con su evidencia:
   - el `PUT /api/productos/{id}/precio`, que no pide autenticación;
   - la constante con la llave en `PasarelaPagosClient` y lo que se imprime en la consola al crear un pedido;
   - las credenciales, la consola H2 y el actuator en `application.properties`.

Paso 6 · Escribir primero la prueba que demuestra el ataque

1. Crea `src/test/java/com/mercaplus/controller/ProductoControllerSeguridadTest.java` con `@WebMvcTest(ProductoController.class)` y `MockMvc`.
2. Reemplaza las dependencias del controlador por mocks con `@MockitoBean`. En el legado el controlador también pide un `EntityManager`: móckealo para que el contexto arranque.
3. Escribe dos pruebas:
   - `inyeccionSqlEsRechazadaConError400`: envía `' OR '1'='1` como `nombre`, espera `400 Bad Request` y verifica que el repositorio nunca se llamó (`verifyNoInteractions`).
   - `busquedaLegitimaFunciona`: envía `cafe` y espera `200`.
4. Ejecuta `mvn test -Dtest=ProductoControllerSeguridadTest`. La primera prueba **debe fallar**: el código actual acepta el ataque. Haz commit con la prueba en rojo.

Paso 7 · Corregir el buscador

1. Elimina el SQL concatenado y el `EntityManager` del controlador.
2. En `ProductoRepository`, agrega una **consulta derivada** de Spring Data que busque por nombre, sin distinguir mayúsculas, y devuelva como máximo 50 resultados. El nombre del método es la consulta; Spring la ejecuta siempre con parámetros, nunca concatenando.
3. Valida el parámetro `nombre` en el controlador con anotaciones de Bean Validation: que no esté vacío, que tenga como máximo 50 caracteres y que solo use una **lista blanca** de caracteres (letras, números, espacio, punto, coma y guion). La dependencia `spring-boot-starter-validation` ya está en el proyecto.
4. Retira el endpoint `PUT /{id}/precio`. Anota en tu tabla STRIDE adónde debería ir esa función y con qué control.
5. Actualiza la prueba: quita el mock del `EntityManager` y ajusta `busquedaLegitimaFunciona` al método nuevo del repositorio.
6. Ejecuta de nuevo `mvn test -Dtest=ProductoControllerSeguridadTest`: las dos pruebas deben pasar. Repite el `curl` del ataque: ahora debe responder `400`. Commit.

No sirve escapar las comillas a mano: la consulta parametrizada separa el dato del SQL siempre.

Paso 8 · Validación en capas

La forma se valida en el adaptador y las reglas en el dominio.

- **Si hiciste el Lab 2**, aplícalo a `/api/v2/pedidos`: el DTO valida la forma (`sku` con un patrón de mayúsculas, números y guion de 3 a 20 caracteres; cantidad de 1 a 100; un máximo de ítems) y responde `400`; el dominio valida las reglas y responde `422`.
- **Si no**, crea un DTO con esas validaciones para `POST /api/pedidos` del legado.
- En los dos casos, escribe una prueba que demuestre que un SKU con caracteres peligrosos se rechaza **antes** de llegar al caso de uso o al servicio.

Paso 9 · Secretos y configuración

1. En `application.properties` reemplaza la URL, el usuario y la contraseña de la base por variables de entorno: `${DB_URL}`, `${DB_USER}` y `${DB_PASSWORD}`.
2. Lee la llave de la pasarela desde la variable `PAGAYA_API_KEY` y elimina la constante del código. Deja de imprimir la llave en la consola.

3. Apaga la consola H2 (`spring.h2.console.enabled=false`) y expone en actuator solo `health` e `info` .
4. Arranca con las variables definidas, por ejemplo: `DB_URL=jdbc:h2:mem:mercaplus DB_USER=app DB_PASSWORD=local PAGAYA_API_KEY=solo-local mvn spring-boot:run` (en Windows, defínelas antes con `set` o en la configuración del IDE).

Paso 10 · Verificar

- [ ] `mvn test` en verde.
- [ ] El ataque responde `400` y la búsqueda de `cafe` sigue funcionando.
- [ ] `PUT /api/productos/1/precio` ya no existe.
- [ ] `localhost:8080/h2-console` no abre la consola.
- [ ] `curl -s localhost:8080/actuator` lista solo `health` e `info` .
- [ ] Al crear un pedido, la consola ya no muestra la llave.
- [ ] `git grep -n "sk_live"` no encuentra nada en el código.
- [ ] Commit final con la etiqueta `lab4` : `git tag lab4` y `git push origin lab4 --tags` .

Entregables y autoevaluación

Entregable	Archivo	Parte
Modelo de amenazas de Threat Dragon	`consultoria/lab4/threat-model.json`	1
Tabla STRIDE con 8 amenazas o más, ordenada por riesgo	`consultoria/lab4/stride.md`	1
Política IAM de mínimo privilegio	`consultoria/lab4/iam-policy.json`	1
Usuario de base de datos	`consultoria/lab4/db-usuario.sql`	1
Código corregido con pruebas de seguridad, etiqueta `lab4`	Tu repositorio	2

Revísate con esta escala por criterio: **4** listo para el cliente · **3** correcto con detalles menores · **2** incompleto pero bien orientado · **1** con errores de fondo · **0** no está.

Criterio	Pregúntate
DFD y STRIDE	¿Las fronteras de confianza están bien ubicadas? ¿Apliqué STRIDE a cada flujo que las cruza? ¿Incluí amenazas que el auditor no mencionó?
Prueba antes de corregir	¿Mi historial de commits muestra la prueba fallando antes de la corrección y pasando después?
Validación en capas	¿La forma está en el adaptador (400) y las reglas en el dominio (422)? ¿Hay una prueba que lo demuestre?
Mínimo privilegio	¿La política IAM no tiene ningún `*`? ¿El usuario de base de datos no puede borrar ni alterar tablas?

Errores frecuentes:

- Corregir la inyección escapando comillas a mano en vez de parametrizar.
- Validar reglas de negocio con anotaciones en el DTO: la forma va en el adaptador y las reglas en el dominio.
- Usar `"Resource": "*"` "temporalmente" en la política IAM.
- Hacer el modelo de amenazas después del código: se pierde su propósito.

Lo que distingue un trabajo sobresaliente: el modelo incluye amenazas que el auditor no reportó, y cada mitigación tiene una prueba o una regla del pipeline que la vigila.

Este laboratorio te prepara para el reto práctico: CrediAndes pide el mismo modelo STRIDE, validación en capas, cero secretos en el repositorio y registros sin datos personales.